Running this provider under ADCS: a readiness plan

Assessment of this provider against what an Active Directory Certificate Services certification authority requires, and the phased plan to close the difference. Generated 2026-10-16 from the markdown of record.

The question. Can this KSP back an Active Directory Certificate Services certification authority — the CA's private key living on a PKCS#11 token, reached through this provider?

The verdict. The architecture is right for it and nothing found so far says otherwise. Today it cannot work, for three reasons that have nothing to do with ADCS, plus four things genuinely missing for the CA case and two that are design decisions rather than code.

This document separates those, because they have completely different costs: one needs a machine, two need a decision from whoever owns the CA's threat model, and four are ordinary engineering.

What a CA actually asks of a KSP

Checked against the source rather than assumed. The point of this table is that the functional surface is largely already there — the problems are elsewhere.

What certsrv needs	State	Evidence
Register under the key-storage interface	**Present**	**tools/register_ksp.ps1** uses **NCRYPT_KEY_STORAGE_INTERFACE** with a user-mode image
Machine-scoped key generation	**Present**	**NCRYPT_MACHINE_KEY_FLAG** honoured in **KSP_CreatePersistedKey**
RSA signing, PKCS#1 v1.5 and PSS	**Present**	verified against two real tokens
ECDSA signing, P-256/384/521	**Present**	verified against two real tokens
Open the CA key by container name	**Present**	**KSP_OpenKey**, 256-byte label budget — ample for a CA name
Enumerate keys	**Present**	**KSP_EnumKeys**, scope-filtered
Store the CA certificate on the key	**Present**	**NCRYPT_CERTIFICATE_PROPERTY** (PROP-14)
Non-exportable private key	**Present, and the default**	**CKA_SENSITIVE** + **CKA_EXTRACTABLE=FALSE**
Advertise algorithms and key sizes	**Half present**	**EnumAlgorithms** yes; **NCRYPT_LENGTHS_PROPERTY missing** — see G3
Tolerate **NCRYPT_SILENT_FLAG**	**Present**	flags are masked with **&**, never compared whole, so **SILENT|PAD_PKCS1** resolves correctly (**p11_utils.c:248**). Benign by construction, but untested — see A4
Survive a multithreaded service	**Present**	32 threads over the 16-session pool, under ThreadSanitizer
Set a DACL on the CA key	**Absent**	**NCRYPT_SECURITY_DESCR_PROPERTY** not implemented — see G6
Refuse a duplicate container name	**Absent**	no **NTE_EXISTS**, no **NCRYPT_OVERWRITE_KEY_FLAG** — see G4
Authenticate without a prompt	**Present but wrong for a CA**	PIN from an environment variable — see G5

The gaps, in the order they bite

G1 — There is no DLL. *Blocking everything.*

ksp_main.c cannot compile without **ncrypt_provider.h**, which declares

NCRYPT_KEY_STORAGE_FUNCTION_TABLE. MSVC compiles the other nine source files clean at **/W3 /WX**; CMake skips the DLL target rather than emitting a provider with no entry point.

Not an ADCS problem. It is *the* problem, and it gates every other phase.

G2 — The binary is unsigned. *Probably blocking.*

tools/sign_ksp.ps1 signs, timestamps and verifies, and CI signs automatically once the certificate secret exists. What is missing is an OV or EV code-signing certificate, which a verified legal entity has to buy.

> **Unverified claim, and it matters:** I expect **ncrypt.dll** to refuse to load an unsigned user-mode KSP. I have not confirmed that and this repository does not either. Settle it in the first hour on Windows (step B1) — if signing is required, it is a procurement lead time, not a code change, and it should start immediately.

G3 — NCRYPT_LENGTHS_PROPERTY is not implemented. *New finding.*

Every shipping KSP implements it. It returns **NCRYPT_SUPPORTED_LENGTHS** (minimum, maximum, increment, default), and it is what the CA setup wizard and **certutil** read to populate and validate key-size choices. The provider answers **NCRYPT_LENGTH_PROPERTY** — the singular, a given key's own size — and has nothing for the plural.

> **The matrix never noticed**, and the reason is instructive: **PROP-02** tracks the singular and there is no row at all for the plural. Two properties one letter apart, one audited and one invisible. This is the same blind spot that hid the two stubs and the missing **NTE_EXISTS**, and the third instance in three sessions.

Cheap to close: the provider already holds **KSP_RSA_MIN_BITS**,

KSP_RSA_MAX_BITS and the step-64 rule, and the EC sizes are per curve.

G4 — A reused container name silently duplicates the key. *Correctness.*

NCryptCreatePersistedKey does not check whether the name is taken and does not handle **NCRYPT_OVERWRITE_KEY_FLAG**, so two token objects can carry the same **CKA_LABEL** and **KSP_OpenKey** returns whichever the token's search hands back first.

For most callers that is untidy. **For a CA it is a correctness failure with certificate-validity consequences**: renewing a CA certificate is exactly the operation that reuses or re-derives a container name, and a CA that signs with an indeterminate key generation issues certificates that may not chain.

Found while making the two-token suite repeatable, two sessions ago, and deferred then as "its own change". The ADCS target promotes it.

Still to confirm: the exact call pattern ADCS uses on renewal — whether it passes **NCRYPT_OVERWRITE_KEY_FLAG**, derives a new name, or expects

NTE_EXISTS. That needs a Windows box (step D4) and it decides whether the fix is a refusal, an overwrite, or both.

G5 — PIN delivery. *A design decision, and the real blocker for production.*

certsrv is a headless service. It never calls

NCryptSetProperty(NCRYPT_PIN_PROPERTY), so the PIN comes from

GetEffectivePin (**p11_session.c:115**), which reads an environment variable. For a service that means a **machine-wide** variable, readable by any process on the host. And **config.h:36** defines a fallback of **"1234"** when nothing is set.

This inverts the security model a CA is bought for. The key cannot be extracted from the token — good — but anyone on that host can ask the token to sign, and for a CA, signing *is* the asset.

Options in Phase C. This one needs an owner's decision, not a patch.

G6 — No ACL on the key. *A design decision.*

`NCRYPT_SECURITY_DESCR_PROPERTY` is not implemented because PKCS#11 has no ACL model (matrix **PROP-11**, Not covered). The Microsoft Software KSP protects machine keys with a filesystem DACL, and the CA MMC's private-key permissions tooling expects to read and set one.

Consequence: you cannot restrict the CA key to the CA's service account through Windows. The only boundary available is the token's own, which Phase D turns into the mitigation.

G7 — The Windows and conformance suites have never executed.

40 integration tests and roughly 150 HLK-conformant checks, written and syntax-checked on Linux only. A suite that has never run is a hypothesis about the suite as much as about the code.

G8 — SoftHSM2 is a software token.

Encrypted SQLite, no hardware protection. Correct for a lab CA; not what you want under a production root or issuing CA. The provider is backend-agnostic, so the production path is a real HSM's PKCS#11 library.

Note the economics honestly: an HSM that ships its own KSP does not need this one. The value here is an HSM *without* a Windows KSP, a token like Kryoptic, or a test environment.

The plan

Five phases. **Phase A is the only one that can be done from this environment**, which is why it comes first — it stops the scarce Windows time being spent on things that did not need Windows.

Phase A — Close what can be closed without Windows

	Work	Why now
A1	Implement `NCRYPT_LENGTHS_PROPERTY` for RSA and each EC curve, as `NCRYPT_SUPPORTED_LENGTHS`. Add the matrix row that should have existed.	G3. The CA wizard reads it; cheap; no Windows needed to get it right
A2	**`NTE_EXISTS`** on a taken container name, plus **`NCRYPT_OVERWRITE_KEY_FLAG`** to replace deliberately. Tests in **both** directions — a check that never fires is as bad as none.	G4. Must land before any renewal test, or that test cannot be trusted
A3	An ADCS-shaped lifecycle suite in **`tests/linux/`**: create a machine-scoped RSA-4096 key, set the certificate property, sign a to-be-signed blob, enumerate, re-create under the same name (must now refuse), overwrite deliberately, delete. Run twice over one token.	De-risks the whole CA flow against a real token before Windows exists. The twice-over pattern is what found the orphaned certificate
A4	A regression test that **`NCRYPT_SILENT_FLAG`** combined with each padding flag still resolves the right mechanism.	The masking is correct today *by construction*; nothing stops a future **`==`** comparison breaking CA signing
A5	Remove the **`"1234"`** PIN fallback for machine scope — **fail closed**. A CA that silently tries a default PIN is worse than one that refuses to start.	G5, the part that needs no decision

Gate: A1–A5 green, fault-injected, and the live suite passing twice.

Phase B — Prove the artifact exists *(needs one Windows machine)*

	Work
B1	Obtain **ncrypt_provider.h** (Cryptographic Provider Development Kit — association unconfirmed; two guesses at its location were already wrong). Build the DLL. **Also settle G2's open question immediately: does an unsigned KSP load at all?**
B2	Register with **register_ksp.ps1**; confirm **certutil -csplist** lists it
B3	Run the 40 integration tests and the ~150 conformance checks. Expect failures; this is their first execution
B4	Confirm the function table layout against the real header, closing **TABLE-01**

Gate: the provider loads, enumerates, and signs through the real **ncrypt.dll**. Until this gate passes, every ADCS statement is a projection.

Phase C — Decide how the CA authenticates to the token

Options, with what each costs. **This needs a decision; I would not pick it silently.**

Option	How	Trade-off
C-a Machine-wide env var	Status quo	Any process on the host reads the PIN. Unacceptable for production; fine for a lab
C-b DPAPI machine-scope blob *(recommended)*	PIN encrypted with **CryptProtectData** at machine scope, stored in a registry value ACL'd to the CA service account; provider decrypts at initialisation	Binds the credential to the host and to one account. Adds a Windows-only code path, so the Linux suites cannot cover it — needs its own Windows test
C-c Protected authentication path	Token asserts **CKF_PROTECTED_AUTHENTICATION_PATH**; no PIN travels through Windows	Cleanest, and only some HSMs offer it. Depends entirely on your token
C-d Token policy: no login for signing	Token configured so the signing key needs no **C_Login**	Moves the control into the HSM, where it arguably belongs. Not possible on SoftHSM2

C-b is the recommendation if the token is SoftHSM2 or any module without C-c. If you are buying an HSM anyway, ask the vendor about C-c first — it removes the problem rather than relocating it.

Phase D — Protect the key without a Windows ACL

The honest position is that this provider cannot offer a DACL, so the boundary has to be the token's.

	Work
D1	**Use per-scope tokens, which already exist.** **KSP_MACHINE_TOKEN_LABEL** + **KSP_MACHINE_PIN** give the CA its own token with its own credential and its own session pool — built in proposal C and verified against two real tokens with different PINs. This is the strongest lever available and it is already implemented
D2	Document plainly that **NCRYPT_SECURITY_DESCR_PROPERTY** is unavailable and that the CA MMC's private-key permissions dialog will not work. A CA operator must not discover that during an incident
D3	Decide whether an unavailable DACL is acceptable for this deployment, or whether it rules out the approach. **This is a risk acceptance, not an engineering task**

Phase E — The ADCS pilot *(needs Windows, after B and A)*

The test plan, in the order that finds problems soonest:

	Test	Watching for
E1	**certutil -csplist** shows the provider; the CA setup wizard offers it and populates key sizes	G3 — the wizard's key-length list is the **NCRYPT_LENGTHS_PROPERTY** test
E2	Install a **standalone test CA** with **-CryptoProviderName "SoftHSM KSP"**, RSA-4096, SHA-256	Key creation under the service account; PIN reaching a headless process
E3	Issue a certificate; publish a CRL	The main signing path through **ncrypt.dll**

	Test	Watching for
E4	**Renew the CA certificate, with a new key**	G4. The test most likely to fail, and the one A2 exists to make meaningful
E5	Restart the CA service; then reboot the host	Lazy initialisation, PIN availability at boot before any logon
E6	Revoke; verify chains with **certutil -verify**	End-to-end correctness, not just API success
E7	Two CAs on one host, or a CA alongside another KSP consumer	The session pool and per-process singleton under a second consumer

Throw the test CA away afterwards. Never pilot against a CA whose certificates anything trusts.

Phase F — Production

Only meaningful once E passes: the code-signing certificate (G2), a real HSM rather than SoftHSM2 (G8), FIPS posture if required (**OPS-02**, needs certified hardware), and HA, which vendor PKCS#11 libraries handle themselves.

Effort, with a warning about these numbers

> **This project's effort estimates were wrong on 4 of the 4 rows it has ever closed.** Two were marked N/A — "no remedy planned" — and were then closed in a day. So read the sizes below as relative, not as a schedule.

Phase	Size	Can it start now?
A	days	**Yes, entirely**
B	hours of work, unknown lead time to obtain the header	No — needs a machine and a header
C	days, plus a decision	Decision now; code after B
D	D1 is done; D2 a page; D3 a decision	Yes
E	days of iteration	No — needs A and B
F	procurement and hardware	Partly now (start the certificate purchase)

The critical path is **the header**, and it has no engineering content. Start looking for it in parallel with Phase A.

What needs a decision from you

1. **Phase C option** — how the CA authenticates. I recommend **C-b (DPAPI)** unless your token supports C-c, in which case take C-c.
2. **Phase D3** — is a CA key with no Windows DACL, protected only by the token boundary and its PIN, acceptable for this deployment?
3. **Target token** — SoftHSM2 (lab only) or a specific HSM? It changes C and G8, and if the HSM ships its own KSP it may change whether this project is the right tool at all.
4. **Signing certificate** — start the purchase now if G2 turns out to be a hard requirement, because it has the longest lead time of anything here.

What this plan does not claim

- That the DLL builds. Nobody has built it.
- That ADCS accepts this provider. No version of it has ever been loaded by **ncrypt.dll**.
- That the ADCS call sequence is fully known. G4's resolution depends on observing it (E4), and some of what is written above about the wizard and renewal is inference from the CNG contract rather than observation.

- That Phase A's work is sufficient. It is what can be established without Windows, which is a different and smaller claim.

Everything above rests on reading this repository's source. Nothing in it has been tested against ADCS, because no artifact exists to test.

`Content fingerprint: a5bb374a7fd9` — of the markdown this was rendered from, plus this script. `tests/check_generated_artifacts.py` recomputes it, so a plan edited without regenerating is a CI failure rather than a PDF that quietly describes an older plan.